

Positional Recoverability in Binary-Addressed Registries

A symmetry obstruction, with a tight worked instance over the finite-type Cartan matrices

Lu Semita
EmergenceByDesign

Version 1.0

Abstract

We formalise a question that arises when a corpus of machine-checkable results is stored in a content-addressed registry laid out over a binary tree: *which structural relations among registry entries can be recovered from their addresses alone, and which must be carried separately as code?* We give a precise definition of positional recoverability relative to a declared predicate vocabulary, and prove an unconditional obstruction: if the vocabulary is symmetric, then no layout, however chosen, and no decoder, however powerful, can recover an antisymmetric relation (Theorem 5.1). The obstruction is a parity mismatch between the addressing primitive and the datum, not a defect of any particular layout, and it yields a lower bound on the irreducible supplement (Corollary 5.3).

We then exhibit a case in which the bound is attained exactly. Taking the registry to hold the Cartan matrices of the finite-dimensional simple Lie algebras, and taking the vocabulary to be tree adjacency alone, we show that every finite-type Dynkin diagram admits a layout in which diagram adjacency coincides with parent–child adjacency of Nock axes (Proposition 6.1), that the sixteen simply-laced types are recovered with *no* supplementary data whatsoever, and that each of the fifteen non-simply-laced types requires exactly one supplementary datum—matching the lower bound in all 31 cases (Theorem 6.2). All claims are machine-checked; the verification code is included and runs in under a second with no dependencies.

The intended reading is methodological rather than Lie-theoretic. The recoverable/-supplementary split is proposed as an *instrument*: one lays a structure into the registry, measures what fails to become positional, and treats that measurement as data about the structure. The present paper establishes that the instrument is well defined, that it returns a sharp answer on a nontrivial and completely classified test family, and—by locating the result inside the existing literature on adjacency labelling schemes—what would and would not be new about it.

1 Scope, and what is not claimed

This is documented independent work. It is not a claim of ownership, priority, or discovery. Every component used here is established, and the relevant literature is surveyed in Section 10 rather than in a footnote, because the honest positioning of the work is part of its content. Specifically:

- The question “can adjacency be decided from vertex labels alone” is the defining question of *adjacency labelling schemes*, introduced by Kannan, Naor and Rudich and independently by Muller in the late 1980s, with antecedents in the 1960s. The present work is a special case of that framework, not an alternative to it.

- Binary addressing with children $2a$ and $2a+1$ is the standard implicit-heap indexing of Williams (1964). Nock’s opcode 0 uses the same arithmetic.
- Content addressing of code by a hash of its syntax tree is the organising principle of the Unison language, and of Merkle-DAG systems generally.
- The classification of finite-type Dynkin diagrams, and every Cartan-matrix fact used as input, is classical.
- Theorem 5.1 is elementary. Its proof is three lines. We state it as a theorem because it is load-bearing—it converts a failed search over layouts into a proof that no layout exists—and not because it is deep. It may well be folklore in the labelling-scheme literature; we have not found it stated in this form, and we do not assert that it is absent.

What we do claim is narrow and, we believe, defensible: (i) the definitions in Section 4 isolate a quantity—the irreducible supplement—that is worth measuring in its own right rather than minimising as an engineering cost; (ii) Theorem 5.1 gives that quantity an unconditional lower bound that is insensitive to layout search; (iii) on the finite-type Cartan matrices the bound is attained exactly in all 31 cases, so the instrument is calibrated against a family where the answer is independently known; and (iv) the arithmetic of Nock axes makes composition of registry addresses a single machine operation (Proposition 3.2), which is what makes the scheme usable for a corpus rather than a single lookup.

2 Motivation: the fragment registry

The setting is a shared corpus of small, individually machine-checkable results—call them *fragments*. A fragment is intended to be checked once and cited thereafter. Three properties are wanted:

- (P1) **Self-naming.** A fragment’s identifier is derived from its content, so that two contributors who derive the same fragment independently obtain the same identifier and discover the duplication without coordination.
- (P2) **Environment-independence.** A fragment means the same thing on every machine, forever. This is where cross-system lemma reuse has historically failed (Section 10).
- (P3) **Cheap recomposition.** Assembling a larger result from fragments must cost little, or the corpus is a catalogue rather than a substrate.

Nock supplies all three cheaply. A noun is an atom (a natural number) or an ordered pair of nouns; a noun’s tree hash gives (P1). The instruction set has twelve opcodes and no undefined behaviour—every formula either yields a value or crashes deterministically—which gives (P2) in a strong form: there is no third outcome in which a fragment quietly means something different. Proposition 3.2 gives (P3).

A fourth property is wanted and is the subject of this paper:

- (P4) **Meaningful addressing.** The identifier should not merely name a fragment but *situate* it, so that relations among fragments are readable from identifiers without consulting a side table.

(P4) is the difference between an addressing schema and a naming convention, and unlike (P1)–(P3) it is not free. Sections 4 and 5 make it precise and show exactly what it costs.

3 Preliminaries

3.1 Nouns and axes

A *noun* is either an *atom* (an element of $\mathbb{N}_{\geq 0}$) or a *cell*, an ordered pair of nouns. Nouns are finite binary trees with atoms at the leaves.

The *axis set* is $\mathbb{A} = \mathbb{N}_{\geq 1}$, carrying the infinite rooted binary tree structure in which 1 is the root and a has children $2a$ (the *head*) and $2a+1$ (the *tail*). Write $d(a) = \lfloor \log_2 a \rfloor$ for the depth of a , so $d(1) = 0$.

The Nock fragment operator $/$ is the partial function

$$\begin{aligned} /[1 \ n] &= n, & /[2 \ (x, y)] &= x, & /[3 \ (x, y)] &= y, \\ /[(2a) \ n] &= /[2 \ /[a \ n]], & /[(2a+1) \ n] &= /[3 \ /[a \ n]], \end{aligned}$$

undefined when a head or tail is taken of an atom. We write $\text{adj}(a, b)$ for *tree adjacency*:

$$\text{adj}(a, b) \iff \lfloor a/2 \rfloor = b \text{ or } \lfloor b/2 \rfloor = a.$$

Let $\pi : \mathbb{A} \rightarrow \{0, 1\}^*$ send a to its binary expansion with the leading 1 deleted; π is a bijection, $\pi(1) = \varepsilon$. Under π , an axis *is* a turn sequence from the root, with 0 meaning head and 1 meaning tail.

3.2 The address monoid

Definition 3.1. For $a, b \in \mathbb{A}$ define $a \circ b = \pi^{-1}(\pi(a) \pi(b))$.

Proposition 3.2 (Composition is one machine operation). $(\mathbb{A}, \circ, 1)$ is a monoid, free on the two generators $\{2, 3\}$, and

$$a \circ b = (a \ll d(b)) \vee (b - 2^{d(b)}) = a \cdot 2^{d(b)} + b - 2^{d(b)}.$$

Moreover, whenever either side is defined,

$$/[(a \circ b) \ n] = /[b \ /[a \ n]]. \tag{1}$$

Proof. π is a bijection onto $\{0, 1\}^*$ and $\circ$ is by definition the transport of concatenation along π ; hence $(\mathbb{A}, \circ, 1)$ is isomorphic to the free monoid $(\{0, 1\}^*, \cdot, \varepsilon)$, whose generators pull back to $\pi^{-1}(0) = 2$ and $\pi^{-1}(1) = 3$. For the closed form: $\pi(a)\pi(b)$ has $\pi(b)$ occupying the low $d(b)$ bits, and $b - 2^{d(b)}$ is exactly $\pi(b)$ read as an integer, so $\pi^{-1}(\pi(a)\pi(b)) = a \cdot 2^{d(b)} + (b - 2^{d(b)})$; the two summands occupy disjoint bit ranges, so $+$ and $\vee$ agree. Equation (1) follows by induction on $|\pi(b)|$ from the recursive clauses of $/$: the base case $b = 1$ is $/[a \ n] = /[1 \ /[a \ n]]$, and the step is the clause for $2b$ or $2b+1$. $\square$ $\square$

Proposition 3.2 is what makes (P3) hold. Nesting one address inside another is a shift and an or; it does not require walking the tree, and the cost of composing k fragments is $O(k)$ machine operations on integers whose width is the sum of the depths.

3.3 Dynkin diagrams

We use only classical facts. To each finite-dimensional complex simple Lie algebra of rank n is associated a Cartan matrix $C = (a_{ij})_{1 \leq i, j \leq n}$ with $a_{ii} = 2$, $a_{ij} \in \mathbb{Z}_{\leq 0}$ for $i \neq j$, and $a_{ij} = 0 \iff a_{ji} = 0$. The *support graph* $G(C)$ has an edge $\{i, j\}$ whenever $a_{ij} \neq 0$. The classification gives the types $A_n, B_n, C_n, D_n, E_6, E_7, E_8, F_4, G_2$; a type is *simply laced* when C is symmetric, equivalently when every edge has $a_{ij}a_{ji} = 1$.

Lemma 3.3 (Classical). *For every finite type, $G(C)$ is a tree of maximum degree at most 3. For every non-simply-laced finite type, C has exactly one unordered pair $\{i, j\}$ with $a_{ij} \neq a_{ji}$.*

Lemma 3.3 is read off the classification; we additionally verify it computationally for all 31 types of rank ≤ 8 in the accompanying code, as a guard against transcription error in our own matrices rather than as a mathematical contribution.

4 Layouts, vocabularies, recoverability

Throughout, V is a finite set of registry entries and W a set of values.

Definition 4.1 (Layout). A *layout* of V is an injection $\lambda : V \rightarrow \mathbb{A}$.

Definition 4.2 (Vocabulary). A *vocabulary* is a family $\Pi = (\pi_i)_{i \in I}$ of functions $\pi_i : \mathbb{A} \times \mathbb{A} \rightarrow D_i$. Π is *symmetric* if $\pi_i(a, b) = \pi_i(b, a)$ for every $i \in I$ and all $a, b \in \mathbb{A}$.

The vocabulary is the declared representation class: it fixes what the schema is permitted to say about a pair of addresses. Examples of symmetric π : tree adjacency adj ; tree distance; depth of the lowest common ancestor; equality; bitwise XOR of the two addresses; whether one address is a suffix-extension of the other *up to order*. Examples of non-symmetric π : the ancestor relation; the signed depth difference; $a - b$; “ a is the head child of b ”.

Definition 4.3 (Positional recoverability). Let $F : V \times V \rightarrow W$. We say F is Π -recoverable under λ if there exists a function $\delta : \prod_{i \in I} D_i \rightarrow W$ with

$$F(u, v) = \delta((\pi_i(\lambda u, \lambda v))_{i \in I}) \quad \text{for all } u, v \in V.$$

No restriction is placed on δ : it need not be computable, efficient, or finitely presented.

Definition 4.4 (Supplement and residue). A *supplement* for F relative to (Π, λ, δ) is a partial function σ on $V \times V$ such that

$$F(u, v) = \begin{cases} \sigma(u, v) & (u, v) \in \text{dom } \sigma, \\ \delta((\pi_i(\lambda u, \lambda v))_i) & \text{otherwise.} \end{cases}$$

The *residue* of F over Π is

$$\text{res}_\Pi(F) = \min_{\lambda, \delta, \sigma} |\{ \{u, v\} : (u, v) \in \text{dom } \sigma \text{ or } (v, u) \in \text{dom } \sigma \}|,$$

the least number of unordered pairs that must be carried outside the schema, minimised over all layouts and all decoders.

$\text{res}_\Pi(F) = 0$ says exactly that F is Π -recoverable under some layout: the addressing schema is meaningful for F in the sense of (P4). $\text{res}_\Pi(F) > 0$ says that some part of F is irreducibly code and not position, and its value counts how much.

5 The symmetry obstruction

Theorem 5.1 (Symmetry obstruction). *Let Π be a symmetric vocabulary. If $F : V \times V \rightarrow W$ is Π -recoverable under some layout λ , then F is symmetric: $F(u, v) = F(v, u)$ for all $u, v \in V$.*

Proof. Fix $u, v \in V$ and let δ witness recoverability. Since each π_i is symmetric, $\pi_i(\lambda u, \lambda v) = \pi_i(\lambda v, \lambda u)$ for every $i \in I$, so the two argument tuples supplied to δ are equal as elements of $\prod_i D_i$. A function takes equal arguments to equal values, hence $F(u, v) = \delta(\cdots) = F(v, u)$. □

Remark 5.2. The strength of Theorem 5.1 is in its quantifiers, not its proof. It holds for *every* layout, *every* decoder including non-computable ones, and *every* symmetric vocabulary including infinite ones. Consequently a failure to recover an antisymmetric relation is not evidence that the search over layouts was insufficiently thorough; the search is provably futile. This is the only reason the statement earns a theorem environment.

Corollary 5.3 (Lower bound on residue). *Let Π be symmetric and let $\text{Asym}(F) = \{\{u, v\} : u \neq v, F(u, v) \neq F(v, u)\}$. Then $\text{res}_\Pi(F) \geq |\text{Asym}(F)|$.*

Proof. Fix any λ, δ, σ . Let $\{u, v\} \in \text{Asym}(F)$ and suppose neither (u, v) nor (v, u) lies in $\text{dom } \sigma$. Then $F(u, v)$ and $F(v, u)$ are both given by δ applied to symmetric data, hence are equal by the argument of Theorem 5.1, contradicting $\{u, v\} \in \text{Asym}(F)$. So every element of $\text{Asym}(F)$ is counted, and the bound holds for each admissible triple; take the minimum. $\square$ $\square$

Corollary 5.3 is the price list. Under any symmetric vocabulary, symmetric structure may be free, but *orientation is never free*: every relation that distinguishes (u, v) from (v, u) costs at least one supplementary datum, and no cleverness in the layout reduces the count.

Remark 5.4 (On enlarging the vocabulary). The obvious response is to admit non-symmetric predicates, and this does defeat Theorem 5.1—trivially so, since the predicate “ λu is the head child of λv ” is antisymmetric and recovers a great deal. The theorem should therefore be read as a statement about a *cost boundary* rather than an impossibility simpliciter: whatever antisymmetric information a schema recovers, it recovers because an antisymmetric primitive was admitted into the vocabulary, and the accounting must record that admission. The methodological point of Section 9 survives vocabulary enlargement, because the residue is always measured relative to a *declared* Π .

6 A tight instance: finite-type Cartan matrices

We now fix $\Pi_0 = \{\text{adj}\}$, the single symmetric predicate of tree adjacency, and take V to be the simple roots of a finite-type Lie algebra with $F = C$ its Cartan matrix.

Proposition 6.1 (Embeddability). *Every connected finite-type Dynkin diagram Δ on $n \geq 1$ vertices admits a layout λ such that for all $u \neq v$,*

$$u \sim_\Delta v \iff \text{adj}(\lambda u, \lambda v).$$

Proof. For $n = 1$ take $\lambda = \{v \mapsto 1\}$. For $n \geq 2$: by Lemma 3.3, Δ is a tree of maximum degree ≤ 3 , and a finite tree on ≥ 2 vertices has at least two leaves. Root Δ at a leaf r and set $\lambda r = 1$. In the resulting rooted tree, r has exactly one child, and every non-root vertex has degree ≤ 3 of which one edge goes to its parent, hence at most two children. Proceeding down the tree, assign the children of a vertex placed at axis a to distinct axes among $\{2a, 2a+1\}$; this is possible since there are at most two children. Injectivity holds because distinct vertices receive distinct turn sequences from r . For the equivalence: each Δ -edge becomes a parent–child pair by construction, giving $\Rightarrow$; conversely if $\text{adj}(\lambda u, \lambda v)$ then one of $\lambda u, \lambda v$ is the parent of the other in the image, and since the image is exactly the rooted tree of Δ , the corresponding vertices are Δ -adjacent. $\square$ $\square$

Theorem 6.2 (Tightness on finite type). *Let C be the Cartan matrix of a finite type of rank n , and let λ be a layout as in Proposition 6.1. Define the positional decoder*

$$\delta_0(u, v) = \begin{cases} 2 & u = v, \\ -1 & \text{adj}(\lambda u, \lambda v), \\ 0 & \text{otherwise.} \end{cases} \quad (2)$$

Then:

type	n	bits	span	edges	positional	residue	axes
A_1	1	1	1	0	0	0	1
A_2	2	2	2	1	1	0	1,2
A_3	3	2	3	2	2	0	2,1,3
A_4	4	3	4	3	3	0	3,1,2,4
A_5	5	3	6	4	4	0	4,2,1,3,6
A_6	6	4	8	5	5	0	6,3,1,2,4,8
A_7	7	4	12	6	6	0	8,4,2,1,3,6,12
A_8	8	5	16	7	7	0	12,6,3,1,2,4,8,16
D_4	4	3	5	3	3	0	1,2,4,5
D_5	5	3	5	4	4	0	3,1,2,4,5
D_6	6	3	6	5	5	0	6,3,1,2,4,5
D_7	7	4	8	6	6	0	8,4,2,1,3,6,7
D_8	8	4	12	7	7	0	12,6,3,1,2,4,8,9
E_6	6	4	8	5	5	0	3,5,1,2,4,8
E_7	7	4	8	6	6	0	8,5,4,2,1,3,6
E_8	8	4	12	7	7	0	10,4,5,2,1,3,6,12
B_2	2	2	2	1	0	1	1,2
B_3	3	2	3	2	1	1	2,1,3
B_4	4	3	4	3	2	1	3,1,2,4
B_5	5	3	6	4	3	1	4,2,1,3,6
B_6	6	4	8	5	4	1	6,3,1,2,4,8
B_7	7	4	12	6	5	1	8,4,2,1,3,6,12
B_8	8	5	16	7	6	1	12,6,3,1,2,4,8,16
C_3	3	2	3	2	1	1	2,1,3
C_4	4	3	4	3	2	1	3,1,2,4
C_5	5	3	6	4	3	1	4,2,1,3,6
C_6	6	4	8	5	4	1	6,3,1,2,4,8
C_7	7	4	12	6	5	1	8,4,2,1,3,6,12
C_8	8	5	16	7	6	1	12,6,3,1,2,4,8,16
F_4	4	3	4	3	2	1	3,1,2,4
G_2	2	2	2	1	0	1	1,2

Table 1: Measured residue for all 31 finite types of rank ≤ 8 under $\Pi_0 = \{\text{adj}\}$. “bits” is the width of the widest address; “span” the largest axis used; “positional” the number of edges recovered by (2) alone. Residue equals $|\text{Asym}(C)|$ in every row. Layouts are those returned by the compactness search of Section 8; they are not canonical, but the residue column is layout-independent by Corollary 5.3 and Theorem 6.2.

(i) if the type is simply laced, δ_0 recovers C exactly and $\text{res}_{\Pi_0}(C) = 0$;

(ii) otherwise $\text{res}_{\Pi_0}(C) = 1$, attained by supplementing the unique pair $\{i, j\}$ with $a_{ij} \neq a_{ji}$.

In both cases $\text{res}_{\Pi_0}(C) = |\text{Asym}(C)|$, so the bound of Corollary 5.3 is attained.

Proof. (i) For a simply-laced type every off-diagonal entry is 0 or -1 , and $a_{ij} = -1$ exactly when $\{i, j\}$ is a Δ -edge, which by Proposition 6.1 holds exactly when $\text{adj}(\lambda i, \lambda j)$; the diagonal is 2 by definition. So δ_0 reproduces C and the empty supplement suffices, giving $\text{res}_{\Pi_0}(C) = 0$; the value cannot be negative.

(ii) By Lemma 3.3 there is exactly one unordered pair with $a_{ij} \neq a_{ji}$, so $|\text{Asym}(C)| = 1$ and Corollary 5.3 gives $\text{res}_{\Pi_0}(C) \geq 1$. For the upper bound, supplement that single pair with $\sigma(i, j) = a_{ij}$, $\sigma(j, i) = a_{ji}$; every remaining off-diagonal entry is 0 or -1 according to Δ -adjacency exactly as in (i), so δ_0 together with σ reproduces C . Hence $\text{res}_{\Pi_0}(C) = 1$. $\square$ $\square$

Table 1 records the machine verification. In total, 965 Cartan entries across 31 algebras were reconstructed and compared against independently transcribed ground truth; all agreed.

7 Fragments as formulas

Positional recoverability concerns relations *among* registry entries. The complementary question is how an entry is *read*, and here Proposition 3.2 does the work.

Lay the registry out as a noun R and let entry v sit at axis λv . The formula $[0 \lambda v]$ evaluated against subject R returns that entry; this is the whole of the read path. If entries are themselves structured—rows of a matrix, say, with columns at axes μ_j inside a row—then reading cell (i, j) is the formula $[0 (\lambda i \circ \mu_j)]$, a single opcode over an address computed by one shift and one or.

Thus a fragment-level result is not merely *stored at* an address; it *is* a formula whose subject is the registry, and composition of results is composition of formulas. Because Nock has no undefined behaviour, a malformed composition crashes rather than returning a wrong value—which is the property (P2) was asking for. In the accompanying code we evaluate all 965 Cartan entries by this route, together with a lemma asserting $a_{ij} = 0 \iff a_{ji} = 0$ expressed entirely as a Nock formula and evaluated for every ordered pair of every algebra.

Remark 7.1 (What Nock does and does not certify). Determinism certifies that a formula applied to a subject yields a particular noun. It does not certify that a stated mathematical claim is true. A registry of fragments therefore requires a checker—a kernel that accepts or rejects a proof object—and that checker is the trusted base of the whole corpus. Nothing in this paper reduces the size of that trusted base; we note the requirement so that it is not mistaken for something the addressing scheme provides.

8 Address width as a measurement

For a layout λ of V put $w(\lambda) = \max_{v \in V} (d(\lambda v) + 1)$, the width in bits of the widest address. Choosing λ to minimise w (then total width, then span) is an optimisation over the finitely many rootings and child-orderings of Δ ; the layouts in Table 1 are its output.

Observation 8.1. w separates structures of equal cardinality. For rank 8: $w = 4$ and span 12 for D_8 and E_8 , but $w = 5$ and span 16 for A_8 , B_8 and C_8 .

The reading we attach to Observation 8.1—that branching compresses and chains do not, so that address width reports on the shape of a diagram and not merely its size—is an interpretation, not a theorem, and we flag it as such. The underlying combinatorial fact is elementary: a path on n vertices embeds in the binary tree only as a zig-zag of depth $\Theta(n)$ when rooted at an endpoint, whereas a branched tree of the same order can be balanced. We record it because it means the compactness objective is not purely cosmetic: minimising address width extracts information rather than merely saving bits.

Remark 8.2 (Not an optimality claim). We emphasise that w is not competitive with the labelling-scheme literature as a measure of representation size. Optimal adjacency labelling for forests is known to $\log_2 n + O(1)$ bits; our scheme is worse, and deliberately so, because the label is required to double as a Nock axis with the composition law of Proposition 3.2. The interest here is in the residue column of Table 1, not the bits column.

9 The residue as an instrument

We can now state the methodological proposal precisely. Given a structure S and a declared vocabulary Π :

1. lay S into the registry, minimising address width;
2. recover what the schema can recover;

3. report $\text{res}_{\Pi}(S)$, together with the identity of the pairs that had to be supplemented.

Step 3 is the output of interest. It is a number attached to S relative to Π , it is bounded below unconditionally by Corollary 5.3, and on the calibration family of Section 6 it takes the value the classification predicts and nothing else. The obstruction is thereby *located* rather than asserted: for F_4 the instrument returns not “this diagram resists positional encoding” but “the pair $\{2, 3\}$, carrying $(-2, -1)$, resists, and nothing else does.”

We stress the direction of inference. A large residue is not a defect of the registry and not a defect of S ; it is a statement about the relationship between S and Π . Changing Π changes the number, which is why Π must be declared before measurement and reported alongside it.

10 Related work

Adjacency labelling schemes. The formal home of Section 4 is the theory of implicit graph representations and adjacency labelling schemes, introduced by Kannan, Naor and Rudich (STOC 1988; SIAM J. Discrete Math. 5(4):596–603, 1992) and independently in the doctoral thesis of Muller (Georgia Tech, 1988), with antecedents in work of Breuer and of Breuer and Folkman in the 1960s. A scheme assigns each vertex a short label such that adjacency is a function of the two labels alone; our Π -recoverability is that condition with the label constrained to be a Nock axis and the decoded value allowed to range over W rather than $\{0, 1\}$. Sharp results are known: adjacency labelling of forests is achievable in $\log_2 n + O(1)$ bits, and the Implicit Graph Conjecture of Kannan, Naor and Rudich—that every hereditary factorial class admits an $O(\log n)$ -bit scheme—was refuted by Hatami and Hatami (FOCS 2022). A reader coming from this literature should regard the present paper as taking the labelling-scheme apparatus and inverting the objective: rather than minimising label length subject to exact recovery, we fix an addressing primitive and measure the exact shortfall in recovery. Distance labelling, routing labelling and other informative labelling schemes form the adjacent literature.

Binary tree addressing. Indexing a binary tree by 1-based integers with children $2a$ and $2a+1$ is the implicit heap layout of Williams (1964). Nock’s opcode 0 (*fragment*) implements exactly this arithmetic over nouns; the address monoid of Proposition 3.2 is the free monoid on two generators under a different name.

Content-addressed code. Unison identifies each definition by a hash of its syntax tree, with human-readable names demoted to metadata, and derives from this its handling of dependency conflicts, caching and code mobility; version 1.0 was released in November 2025. Merkle DAGs, Git, Nix and IPFS instantiate the same principle for other artefact classes. Property (P1) of Section 2 is not a contribution of this work; it is inherited.

Cross-system lemma reuse. The motivating failure mode of Section 2—the same theorem proved repeatedly in mutually unintelligible systems—is documented and attacked in the proof-interchange literature. OpenTheory provides an interchange format for the HOL family; Dedukti, a logical framework based on the $\lambda\Pi$ -calculus modulo rewriting, has been used as a universal proof checker and as a hub for translating libraries between Coq, Lean, Matita, PVS and OpenTheory, with Fermat’s little theorem as a worked end-to-end example. Large curated libraries (Mizar’s MML, Isabelle’s Archive of Formal Proofs, Lean’s `mathlib`) are the practical state of the art in lemma reuse within a single logic. The present paper addresses none of the logical translation problems these systems solve; it concerns only the addressing layer beneath them.

Nock, NockApp, Nockchain. Nock is the combinator instruction set underlying Urbit. The NockApp framework pairs a Rust runtime hosting a Nock evaluator and I/O drivers with a kernel—a noun—holding provable application logic, and Nockchain uses a STARK proving system over Nock execution traces as its consensus work. We use Nock only as a specification of the addressing arithmetic and of deterministic evaluation; nothing in this paper depends on Nockchain, and in particular no claim here requires the zero-knowledge lock-script opcode, which at the time of writing is announced but not shipped.

Minimal description length. Choosing a canonical form by shortest description is Kolmogorov complexity in outline, and is not computable in general. Our Section 8 optimisation is not a description-length minimisation: it searches a small finite set of layouts under a fixed encoding.

11 Limitations and open residue

We list the gaps explicitly, in the order in which we would attack them.

- (L1) **The calibration family is small and atypical.** The finite Dynkin diagrams are trees of maximum degree 3, which is precisely the condition making Proposition 6.1 easy. Nothing here shows the instrument behaves well on families that are not near-tree. The honest summary is that the instrument has been calibrated, not tested.
- (L2) **Non-tree structures are unaddressed.** Root systems, subalgebra inclusion lattices and dependency graphs of a real fragment corpus are directed acyclic graphs, not trees. A tree layout must then either duplicate vertices or abandon uniqueness of the shortest address. The natural move—a spanning “spine” carrying canonical addresses with all remaining edges as formulas—is stated here as a direction, with the cost measured the same way; it is not carried out.
- (L3) **No canonical form is established.** We have not shown that minimising address width yields a unique layout, and in general it does not: Table 1 exhibits several types sharing an optimal width. A registry needs an invariant name (a content hash) distinct from the working address, and a verifiable account of re-layout when usage statistics shift. Both are designed but not implemented.
- (L4) **Polynomials are not treated.** A registry of algebraic identities at polynomial granularity is the motivating application, and no result in this paper bears on it. The Cartan matrices are integer combinatorial data; whether the residue instrument says anything useful about, say, a basis of relations among polynomials is open, and we regard it as the first real test rather than a corollary.
- (L5) **The checker is the trusted base and is not built.** See the remark closing Section 7. Until a proof-object checker exists, a “fragment registry” is an addressing scheme for claims, not for theorems.
- (L6) **Vocabulary dependence.** Theorem 5.1 constrains symmetric vocabularies. We conjecture, but do not prove, that for the finite-type Cartan matrices the residue remains $|\text{Asym}(C)|$ under every vocabulary generated by the symmetric predicates listed in Section 4 together with Boolean combination; and we note that this conjecture is refuted the moment a single antisymmetric primitive is admitted, so its interest lies entirely in the choice of Π .

12 Reproducibility

All claims labelled as verified were checked by the accompanying code: `axis.py` (address arithmetic and its self-tests), `dynkin.py` (independently transcribed Cartan matrices, with validity and tree checks), `probe.py` (layout search, reconstruction, residue), `lemma.py` (a Nock evaluator for opcodes 0–9, conformance-tested against the specification’s own examples including the recursive decrement gate, and used to read every Cartan entry by formula), `test_symmetry.py` (the standing tests corresponding to Corollary 5.3 and Theorem 6.2), and `render.py` (figures). There are no dependencies beyond the Python standard library and the full suite completes in under one second. A self-contained browser demonstrator reproduces Theorem 6.2 interactively.

The single check we regard as decisive is `test_residue_equals_asymmetric_edges`: for all 31 types, the set of pairs requiring supplementation is exactly $\text{Asym}(C)$, with no exceptions and no fitted parameters.

Lu Semita · EmergenceByDesign. Documented independent work. This document asserts no priority, ownership, or discovery; prior art is surveyed in Section 10 and all classical inputs are attributed there. Corrections and refutations are welcomed, in particular against (L1), (L4) and (L6).